HDOC Day-56 Activity

First Negative Integer in Each Sliding Window Using Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-56

Aim

To write a C program that considers every contiguous subarray of size **k** as a sliding window and displays the first negative integer present in each window. The array is passed to **first_negative()** using its address and accessed using pointers.

Problem Requirements

- Read the size and elements of a variable-length integer array **arr[]**.
- Read an integer **k** such that 0 < k <= size.
- Pass the array to **first_negative()** using its address.
- Consider every contiguous group of k elements as a sliding window.
- Find the first negative integer in each window from left to right.
- Display **0** if a window contains no negative integer.
- Separate results by spaces and display **-1** when k is invalid.
- Use a nested-loop approach.

Algorithm

1. Read array size **n**, the array elements, and **k**.
2. Pass **arr**, **n**, and **k** to **first_negative()**.
3. If **k <= 0** or **k > n**, print -1 and return.
4. Use an outer loop from **i = 0** to **n-k**; each i is the beginning of a sliding window.
5. Set **negative = 0** for the current window.
6. Use an inner loop from j = 0 to k-1. If ***(arr + i + j) < 0**, store it in negative and immediately break.
7. Print negative. If none was found, its value remains 0.

C Program

```c
#include <stdio.h>

void first_negative(int *arr, int n, int k)
{
    if (k <= 0 || k > n)
    {
        printf("-1");
        return;
    }

    for (int i = 0; i <= n - k; i++)
    {
        int negative = 0;

        for (int j = 0; j < k; j++)
        {
            if (*(arr + i + j) < 0)
            {
                negative = *(arr + i + j);
                break;
            }
        }

        printf("%d", negative);

        if (i < n - k)
        {
            printf(" ");
        }
    }
}

int main()
{
    int n, k;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    first_negative(arr, n, k);

    return 0;
}
```

Pointer Usage

first_negative(arr, n, k) passes the address of the first array element. The parameter **int *arr** receives that address. The expression ***(arr + i + j)** accesses each element in the current sliding window.

Why break is Used

The window must return the **first** negative integer, not every negative integer. Therefore, as soon as a negative value is found while scanning the window from left to right, **break** stops the inner loop.

Example 1 Dry Run

Window	Elements	First Negative
1	12 -1 -7	-1
2	-1 -7 8	-1
3	-7 8 -15	-7
4	8 -15 30	-15
5	-15 30 16	-15
6	30 16 28	0

Therefore, the output is **-1 -1 -7 -15 -15 0**.

Test Cases / Expected Results

Case	Input Array	k	Expected Output
Example 1	12 -1 -7 8 -15 30 16 28	3	-1 -1 -7 -15 -15 0
Example 2	1 2 -3 4 5 -6	2	0 -3 -3 0 -6
Example 3	1 2 3 4	2	0 0 0
Example 4	1 2 3	4	-1

Example 2 Explanation

For array **1 2 -3 4 5 -6** and k = 2, the windows are [1,2], [2,-3], [-3,4], [4,5], and [5,-6]. Their first negative values are respectively **0, -3, -3, 0, -6**.

Example 3 Explanation

For array **1 2 3 4** and k = 2, none of the three windows contains a negative number, so the output is **0 0 0**.

Example 4 - Invalid k

The array size is 3 but k = 4. Since k is greater than the array size, it is invalid and the program displays **-1**.

Complexity

Time Complexity: O((n-k+1) x k), generally O(nk), because the nested-loop approach may examine all k elements of every window.

Space Complexity: O(1) extra space because no additional array is required.

Result

The program successfully finds the first negative integer in every contiguous sliding window of size k using pointers and nested loops. It prints 0 when a window has no negative integer and -1 when k is invalid.

Execution Example

```
Enter the size of array: 8
Enter the elements: 12 -1 -7 8 -15 30 16 28
Enter k: 3
-1 -1 -7 -15 -15 0
```